CMP 519 Software Engineering

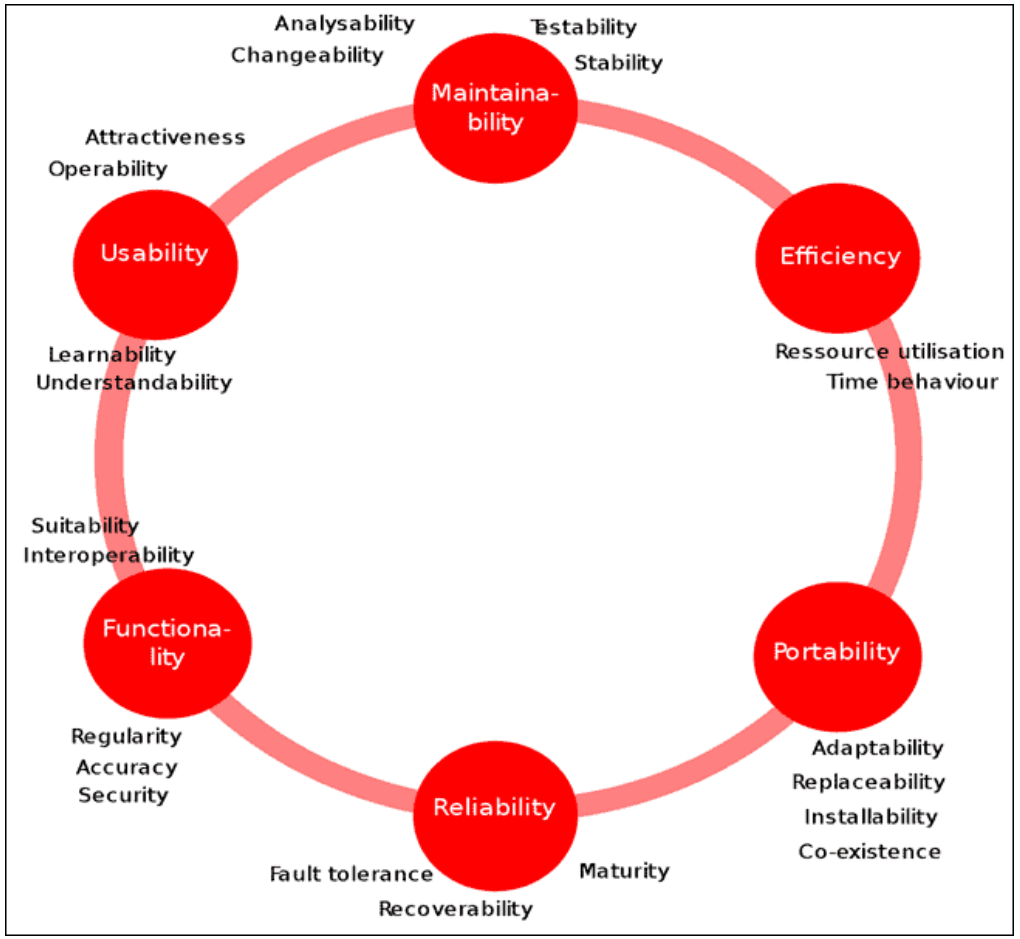
UNIT 06 – Software Quality Assurance and Testing

Instructor: Dr. Suresh Pokharel

DISCLAIMER

These slides are part of teaching materials for Software Engineering. These slides do not cover all aspect of learning Software Engineering, nor are these be taken as primary source of information. As the core textbooks and reference books for learning the subject has already been specified and provided to the students, students are encouraged to learn from the original sources.

Software Quality Concepts and Attributes



SOFTWARE QUALITY

- The degree to which a system, component, or process meets specified requirements. - by Philip Crosby
- The degree to which a system, component, or process meets customer or user needs or expectations. - by Joseph M. Juran

SOFTWARE QUALITY

Pressman believes that Software quality is :

- Conformance to explicitly stated functional and performance requirements, (meets the specs)
  - Discuss: 'functional' and 'performance' specs!!

- Explicitly documented development standards, and

  seems to imply a documented development process

- Implicit characteristics that are expected of all professionally developed software.

  further seems to imply quality as found in reliability, maintainability, scalability, usability, and more

WHAT ARE SQA, SQP, SQC AND SQM?

SQA includes all 4 elements…

1. **Software Quality Assurance** – establishment of network of organizational procedures and standards leading to high-quality software
2. **Software Quality Planning** – selection of appropriate procedures and standards from this framework and adaptation of these to specific software project
3. **Software Quality Control** – definition and enactment of processes that ensure that project quality procedures and standards are being followed by the software development team
4. **Software Quality Metrics** – collecting and analyzing quality data to predict and control quality of the software product being developed

CLASSIFICATION OF SOFTWARE QUALITIES

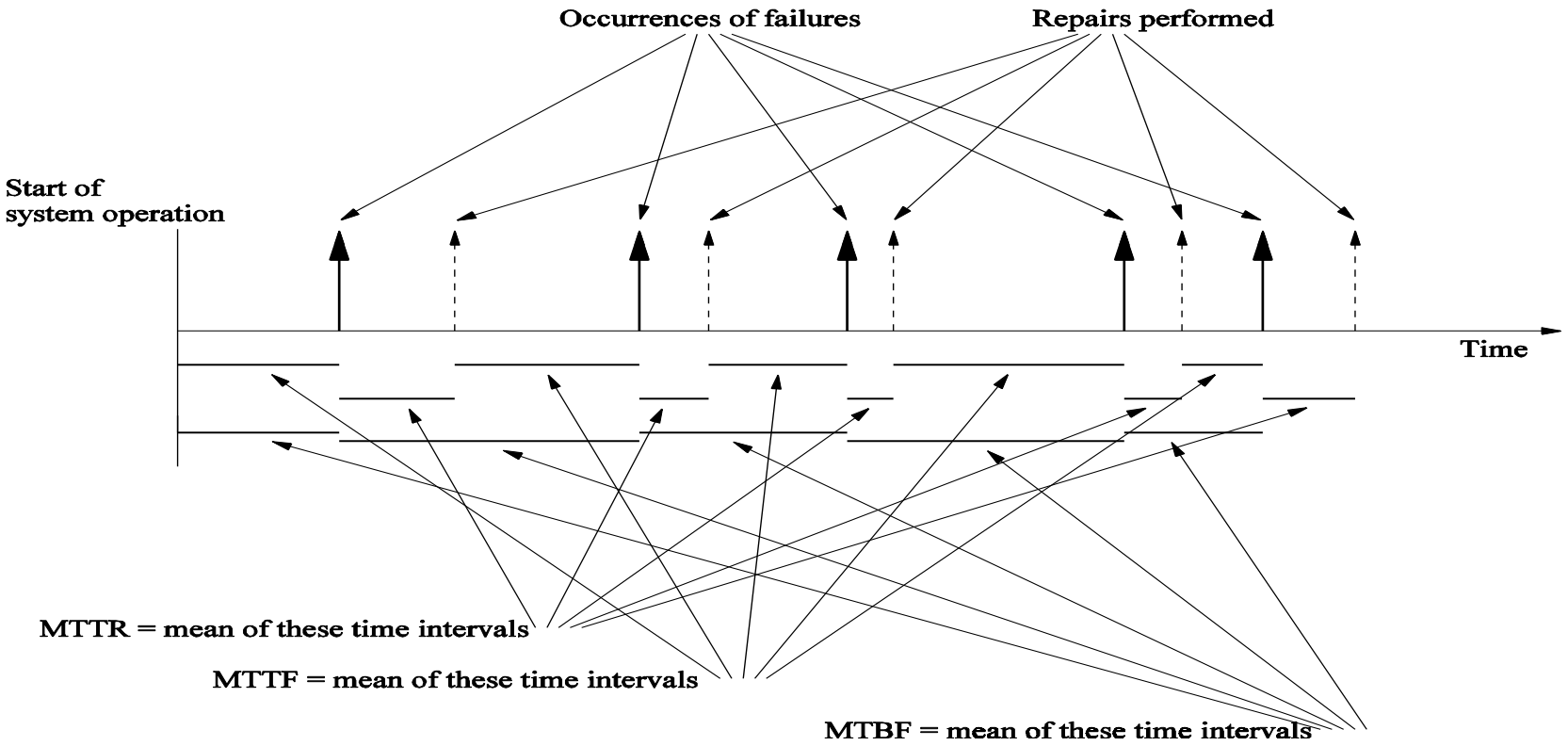
- External qualities
  - Visible to user of system
- Internal qualities
  - Concern with developers of system

RELIABILITY

- Measures the software's ability to maintain performance under specific conditions for a defined period
- Key metrics include Mean Time Between Failures (MTBF) and Mean Time To Repair (MTTR)
- Includes fault tolerance, recoverability, and consistency in producing expected results
- Can be improved through rigorous testing, error handling, and proper exception management
- Critical for mission-critical systems and safety-critical applications

WHAT IS RELIABILITY?

- MTTF: Mean Time To Failure
- MTTR: Mean Time To Repair
- MTBF: Mean Time Between Failures (= MTTF + MTTR)
- Steady State Availability (A) = MTTF/(MTTF + MTTR) = MTTF/MTBF



Relationship between MTTR, MTTF, and MTBF.

Sagar Naik (University of Waterloo) and Piyu Tripathy (NEC Labs of America), Software Testing and QA

USABILITY

- Focuses on how easily users can learn, operate, and interact with the software
- Encompasses user interface design, accessibility, and overall user experience
- Key aspects include:
  - Learnability: How quickly new users can become proficient
  - Efficiency: Speed at which experienced users can accomplish tasks
  - Memorability: How easily users can return after a period of non-use
  - Error prevention and recovery: Minimizing user mistakes and helping users recover
  - User satisfaction: Overall positive experience with the software

EFFICIENCY

- Measures how well the software utilizes system resources
- Important aspects include:
  - Time behavior: Response time, processing time, throughput rates
  - Resource utilization: CPU usage, memory consumption, disk space, network bandwidth
  - Scalability: Ability to handle increased workload without degradation
- Can be optimized through performance tuning and algorithmic improvements
- Must balance efficiency with other quality attributes like maintainability

MAINTAINABILITY

- The term "software maintenance" is commonly used to refer to the modifications that are made to a software system after its initial release.
- There is evidence that maintenance costs exceed 60% of the total costs of the software.

MAINTAINABILITY

- Reflects how easily software can be modified, updated, and enhanced
- Key components include:
  - *Modularity*: Organization of code into logical, independent components
  - *Readability*: Clear, well-documented code that others can understand
  - *Testability*: Ease of testing changes and modifications
  - *Flexibility*: Ability to adapt to new requirements or environments
  - *Code reusability*: Potential for code components to be reused
- Enhanced through good documentation, coding standards, and architectural design patterns

SOFTWARE SAFETY

- Focuses on preventing software from causing harm to users, environment, or property
- Critical in systems where software failure could lead to life-threatening situations
- Key aspects:
  - *Hazard Analysis*: Identifying potential safety risks early in development
  - *Fault Detection and Prevention*: Building mechanisms to detect and prevent dangerous conditions
  - *Fail-Safe Systems*: Ensuring system fails in a safe, controlled manner
  - *Safety Certification*: Meeting industry safety standards and regulations
  - *Real-time Monitoring*: Continuous monitoring of critical safety parameters

SOFTWARE SECURITY

- Protects software from unauthorized access, attacks, and vulnerabilities
- Security Requirements
  - Authentication: Verifying user identity
  - Authorization: Managing access rights
  - Confidentiality: Protecting sensitive data
  - Integrity: Ensuring data remains unaltered
  - Availability: Maintaining system accessibility
  - Non-repudiation: Preventing denial of actions taken

- Common Security Threats
  - Malware: Viruses, worms, trojans, ransomware
  - SQL Injection: Attacking database through malicious queries
  - Cross-Site Scripting (XSS): Injecting malicious scripts
  - Buffer Overflow: Exploiting memory boundaries
  - Man-in-the-Middle Attacks: Intercepting communications
  - Denial of Service (DoS): Overwhelming system resources

SOFTWARE SECURITY

- Security Measures
  - Input Validation: Checking all user inputs for validity
  - Encryption: Protecting data in transit and at rest
  - Access Control: Implementing proper authorization mechanisms
  - Regular Updates: Patching security vulnerabilities
  - Security Testing: Penetration testing and vulnerability assessments
  - Secure Coding Practices: Following security guidelines in development

- Security Best Practices
  - Defense in Depth: Multiple layers of security controls
  - Principle of Least Privilege: Minimal access rights needed
  - Regular Security Audits: Systematic evaluation of security measures
  - Incident Response Planning: Procedures for handling security breaches
  - Security Training: Educating developers and users
  - Compliance: Meeting regulatory requirements

SOFTWARE SECURITY

- Integration of Safety and Security
  - Safety and security often overlap and complement each other
  - Both require:
    - Risk Assessment: Identifying potential threats and hazards
    - Systematic Design: Building protection mechanisms into software
    - Regular Testing: Verifying effectiveness of safety and security measures
    - Documentation: Maintaining records of all safety and security measures
    - Continuous Monitoring: Tracking system behavior and potential issues

- Future Considerations
  - Emerging technologies like AI and IoT bringing new safety and security challenges
  - Growing importance of cybersecurity in connected systems
  - Increasing regulatory requirements for safety and security
  - Need for integrated safety and security approaches
  - Evolution of threat landscapes requiring adaptive security measures

SOFTWARE QUALITY ASSURANCE (SQA)

- Software quality assurance (often called quality management) is an umbrella activity that is applied throughout the software process.
- Software quality assurance encompasses a broad range of concerns and activities that focus on the management of software quality.
- Software quality assurance (SQA) encompasses:
  - An SQA process,
  - Specific quality assurance and quality control tasks (including technical reviews and a multi-tiered testing strategy)
  - Effective software engineering practice (methods and tools)
  - Control of all software work products and the changes made to them
  - A procedure to ensure compliance with software development standards (when applicable)
  - Measurement and reporting mechanisms

ELEMENTS OF SOFTWARE QUALITY ASSURANCE

1. Standards
2. Reviews and audits
3. Testing
4. Error/defect collection and analysis
5. Change management
6. Education
7. Vendor management
8. Security management
9. Safety
10. Risk management

ELEMENTS OF SOFTWARE QUALITY ASSURANCE

1. Standards:

The job of SQA is to ensure that standards that have been adopted are followed and that all work products conform to them.

2. Reviews and audits:

Technical reviews are a quality control activity. Their intent is to uncover errors.

Audits are a type of review performed by SQA personnel with the intent of ensuring that quality guidelines are being followed for software engineering work.

3. Testing:

The job of SQA is to ensure that testing is properly planned and efficiently conducted so that it has the highest likelihood of achieving its primary goal.

ELEMENTS OF SOFTWARE QUALITY ASSURANCE

4. Error/defect collection and analysis:

SQA collects and analyzes error and defect data to better understand how errors are introduced and what software engineering activities are best suited to eliminating them.

5. Change management:

SQA ensures that adequate change management practices have been instituted.

6. Education:

The SQA organization takes the lead in software process improvement and is a key proponent and sponsor of educational programs.

7. Vendor management:

The job of the SQA organization is to ensure that high-quality software results by suggesting specific quality practices that the vendor should follow (when possible), and incorporating quality mandates as part of any contract with an external vendor.

ELEMENTS OF SOFTWARE QUALITY ASSURANCE

8. Security management:

SQA ensures that appropriate process and technology are used to achieve software security.

9. Safety:

SQA may be responsible for assessing the impact of software failure and for initiating those steps required to reduce risk.

10. Risk management:

Although the analysis and mitigation of risk is the concern of software engineers, the SQA organization ensures that risk management activities are properly conducted and that risk-related contingency plans have been established.

SQA TASKS

- **Prepares an SQA plan for a project**

  The plan identifies evaluations to be performed, audits and reviews to be conducted, standards that are applicable to the project, procedures for error reporting and tracking, work products that are produced by the SQA group.

- **Participates in the development of the project's software process description**

  The SQA group reviews the process description for compliance with organizational policy, internal software standards, externally imposed standards (e.g., ISO-9001), and other parts of the software project plan.

- **Review software engineering activities to verify compliance with the defined software process**

  The SQA group identifies, documents, and tracks deviations from the process and verifies that corrections have been made.

SQA TASKS

- **Audits designated software work products to verify compliance with those defined as part of the software process**

  The SQA group reviews selected work products; identifies, documents, and tracks deviations; verifies that corrections have been made; and periodically reports the results of its work to the project manager.

- **Ensures that deviations in software work and work products are documented and handled according to a documented procedure**

  Deviations may be encountered in the project plan, process description, applicable standards, or software engineering work products.

- **Records any noncompliance and reports to senior management**

  Noncompliance items are tracked until they are resolved.

SQA: GOALS, ATTRIBUTES AND METRICS

Goals

- Requirement quality
- Design quality
- Code quality
- Quality control effectiveness

SQA: GOALS, ATTRIBUTES AND METRICS

Goal	Attribute	Metric
Requirement quality	Ambiguity	Number of ambiguous modifiers (e.g., many, large, human-friendly)
	Completeness	Number of TBA, TBD
	Understandability	Number of sections/subsections
	Volatility	Number of changes per requirement
		Time (by activity) when change is requested
	Traceability	Number of requirements not traceable to design/code
	Model clarity	Number of UML models
		Number of descriptive pages per model
		Number of UML errors
Design quality	Architectural integrity	Existence of architectural model
	Component completeness	Number of components that trace to architectural model
		Complexity of procedural design
	Interface complexity	Average number of pick to get to a typical function or content
		Layout appropriateness
	Patterns	Number of patterns used

SQA: GOALS, ATTRIBUTES AND METRICS

Goal	Attribute	Metric
Code quality	Complexity	Cyclomatic complexity
	Maintainability	Design factors (Chapter 8)
	Understandability	Percent internal comments
		Variable naming conventions
	Reusability	Percent reused components
	Documentation	Readability index
QC effectiveness	Resource allocation	Staff hour percentage per activity
	Completion rate	Actual vs. budgeted completion time
	Review effectiveness	See review metrics (Chapter 14)
	Testing effectiveness	Number of errors found and criticality
		Effort required to correct an error
		Origin of error

Role of QA in Software Lifecycle

Lifecycle Phase	QA Activities	Deliverables
Requirements Phase	Quality planning and standards definition	Quality Management Plan
	Requirements review and validation	Requirements Review Report
	Traceability matrix creation	Traceability Matrix
	Risk identification	Initial Risk Assessment
	Quality metrics definition	Quality Metrics Document
Design Phase	Architecture review	Design Review Reports
	Design validation	Security Assessment Document
	Interface design review	Performance Requirements
	Security design assessment	Design Standards Compliance Report
	Performance considerations	Technical Documentation Review
Development Phase	Code review process	Code Review Reports
	Coding standards enforcement	Static Analysis Results
	Unit testing oversight	Unit Test Results
	Static code analysis	Security Test Reports
	Security testing	Development Documentation
Testing Phase	Test planning and strategy	Test Plan
	Test case development	Test Cases
	Various testing execution	Test Results
	Defect tracking	Defect Reports
	Test metrics collection	Test Metrics Report
Deployment Phase	Release readiness assessment	Release Readiness Report
	Installation testing	Installation Guide
	Configuration verification	Configuration Manual
	Performance monitoring	Performance Baseline
	User documentation review	User Documentation
Maintenance Phase	Change request assessment	Change Assessment Reports
	Impact analysis	Impact Analysis Documents
	Regression testing	Regression Test Results
	Bug fix verification	Bug Fix Verification Reports
	User feedback analysis	User Feedback Analysis

PROJECT ESTIMATION

- Project scope must be understood
- Elaboration (decomposition) is necessary
- Historical metrics are very helpful
- At least two different techniques should be used
- Uncertainty is inherent in the process

ESTIMATION ACCURACY

- The degree to which the planner has properly estimated the size of the product to be built
- The ability to translate the size estimate into human effort, calendar time, and $$ (a function of the availability of reliable software metrics from past projects)
- The degree to which the project plan reflects the abilities of the software team
- The stability of product requirements and the environment that supports the software engineering effort

ESTIMATION TECHNIQUES

- **Wait till the end!!**
- **Past (similar) project experience**
- **Conventional estimation techniques/ Decomposition**
  - task breakdown and effort estimates
  - size (e.g., Function Points) estimates
- **Empirical models**
- **Automated tools**

CONVENTIONAL METHOD – LINE OF CODE(LOC)

- Decompose the project into different functions
- Estimate LOC for each function
- For estimation of LOC, a three-point

  or expected value can be computed as below

$$S = \frac{s_{\text{opt}} + 4s_m + s_{\text{pess}}}{6}$$

Where, S: expected value, S_{opt}: Optimistic, S_m: most likely, S_{pess}: Pesimistic

Historical Data:

- Average productivity: 620 LOC/pm
- Manpower cost: $8,000 per month
- **Total manpower : 33,200/620 = 54 person-month**
- **Total cost: $432,000**

Function	Estimated LOC
User interface and control facilities (UICF)	2,300
Two-dimensional geometric analysis (2DGA)	5,300
Three-dimensional geometric analysis (3DGA)	6,800
Database management (DBM)	3,350
Computer graphics display facilities (CGDF)	4,950
Peripheral control function (PCF)	2,100
Design analysis modules (DAM)	8,400
Estimated lines of code	*33,200*

An Example

CONVENTIONAL METHOD – FUNCTION POINTS (FP)

- Count the number of information domain value
- Compute the Function Point(FP) using below formula

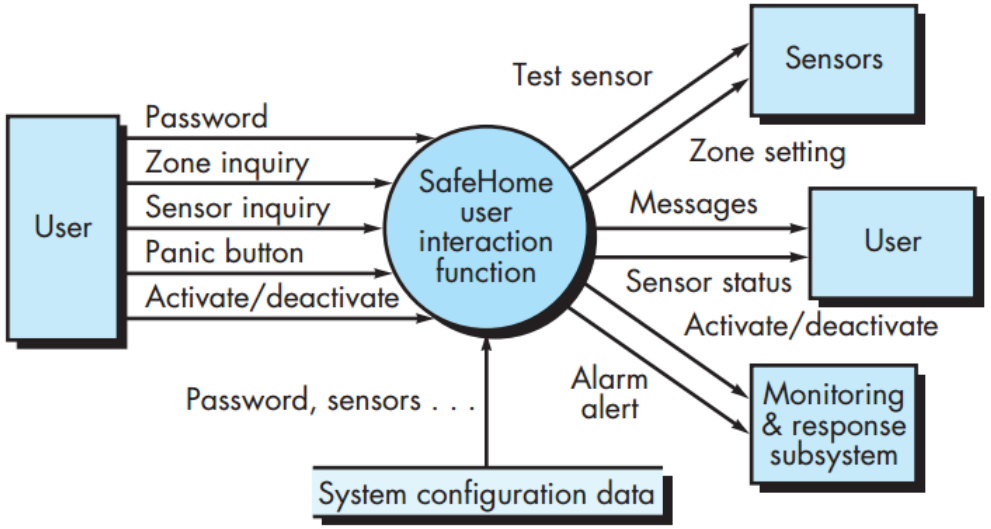
$$FP = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)]$$

F_i (i 1 to 14) are *value adjustment factors* (VAF) and related to the following factors.

Information Domain Value	Count		Weighting factor Simple	Average	Complex		
External Inputs (EIs)		3	3	4	6	=	
External Outputs (EOs)		3	4	5	7	=	
External Inquiries (EQs)		3	3	4	6	=	
Internal Logical Files (ILFs)		3	7	10	15	=	
External Interface Files (EIFs)		3	5	7	10	=	
Count total							

1. Does the system require reliable backup and recovery?
2. Are specialized data communications required to transfer information to or from the application?
3. Are there distributed processing functions?
4. Is performance critical?
5. Will the system run in an existing, heavily utilized operational environment?
6. Does the system require online data entry?
7. Does the online data entry require the input transaction to be built over multiple screens or operations?
8. Are the ILFs updated online?
9. Are the inputs, outputs, fi les, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and installation included in the design?
13. Is the system designed for multiple installations in different organizations?
14. Is the application designed to facilitate change and ease of use by the user?

CONVENTIONAL METHOD – FUNCTION POINTS (FP)



Information Domain Value	Count		Weighting factor Simple	Average	Complex		
External Inputs (EIs)	3	3	3	4	6	=	9
External Outputs (EOs)	2	3	4	5	7	=	8
External Inquiries (EQs)	2	3	3	4	6	=	6
Internal Logical Files (ILFs)	1	3	7	10	15	=	7
External Interface Files (EIFs)	4	3	5	7	10	=	20
Count total							50

EIs: password, panic button, and activate/deactivate
EQs: zone inquiry and sensor inquiry
ILF: system configuration file
EOs: messages and sensor status
EIFs: test sensor, zone setting, activate/deactivate, and alarm alert

$$FP = \text{count total} \times [0.65 + 0.01 \times \Sigma(F_i)]$$

$$FP = 50 \times [0.65 + (0.01 \times 46)] = 56$$

- organizational average productivity = 6.5 FP/pm.
- burdened labor rate = $8000 per month, approximately $1230/FP.
- Based on the FP estimate and the historical productivity data, **total estimated project cost is $68,880 and estimated effort is 9 person-months.**

FP - Exercise

Question: Calculate the function point, total estimated project cost and estimated effort for software application with multiple Processing Factors 5, 1, 0, 4, 3, 5, 4, 3, 4, 5, 2, 3, 4, 2 by using following given Date: The number of EI(Avg): 22,The number of EO(Low): 45,The number of EI(High): 06, The number of ILF(Avg): 05, The number of ELF(Low): 02 given that - average productivity = 6.5 FP/pm and burdened labor rate = $8000 per month.

Parameters	Weight Factors		
	Low	Average	High
External Inputs (EI)	3	4	6
External Outputs (EO)	4	5	7
External Inquiries (EI)	3	4	6
Internal Logic Files (ILF)	7	10	15
External Logic Files(ELF)	5	7	10

PROCESS-BASED ESTIMATION

Activity →	CC	Planning	Risk analysis	Engineering		Construction release		CE	Totals
Task →				Analysis	Design	Code	Test		
Function									
↓									
UICF				0.50	2.50	0.40	5.00	n/a	8.40
2DGA				0.75	4.00	0.60	2.00	n/a	7.35
3DGA				0.50	4.00	1.00	3.00	n/a	8.50
CGDF				0.50	3.00	1.00	1.50	n/a	6.00
DBM				0.50	3.00	0.75	1.50	n/a	5.75
PCF				0.25	2.00	0.50	1.50	n/a	4.25
DAM				0.50	2.00	0.50	2.00	n/a	5.00
Totals	0.25	0.25	0.25	3.50	20.50	4.50	16.50		46.00
% effort	1%	1%	1%	8%	45%	10%	36%		

CC = customer communication CE = customer evaluation

Average burdened labor rate of $8,000 per month

Estimated effort is 46 person-months

Total Cost: $368,000

EMPERICAL MODELS – THE STRUCTURE OF ESTIMATION MODEL

$$E = A + B \times (e_v)^C$$

where *A, B,* and *C* are empirically derived constants, *E* is effort in person-months, and e_v is the estimation variable (either LOC or FP).

EMPERICAL MODELS – COCOMO MODEL

Refer to separate slide

TESTING FUNDAMENTALS

To find errors

Testing is the process of exercising a program with the specific intent of finding errors prior to delivery to the end user.

CHARACTERISTICS OF TESTABLE SOFTWARE

- **Operable** - The better it works (i.e., better quality), the easier it is to test
- **Observable** –Incorrect output is easily identified; internal errors are automatically detected
- **Controllable** –The states and variables of the software can be controlled directly by the tester
- **Decomposable** –The software is built from independent modules that can be tested independently

CHARACTERISTICS OF TESTABLE SOFTWARE

- **Simple** –The program should exhibit functional, structural, and code simplicity
- **Stable** –The fewer the changes, the fewer the disruptions to testing
- **Understandable** –The architectural design is well understood; documentation is available and organized

TEST CHARACTERISTICS

A good test has a high probability of finding an error

- The tester must understand the software and how it might fail

A good test is not redundant

- Testing time is limited; one test should not serve the same purpose as another test

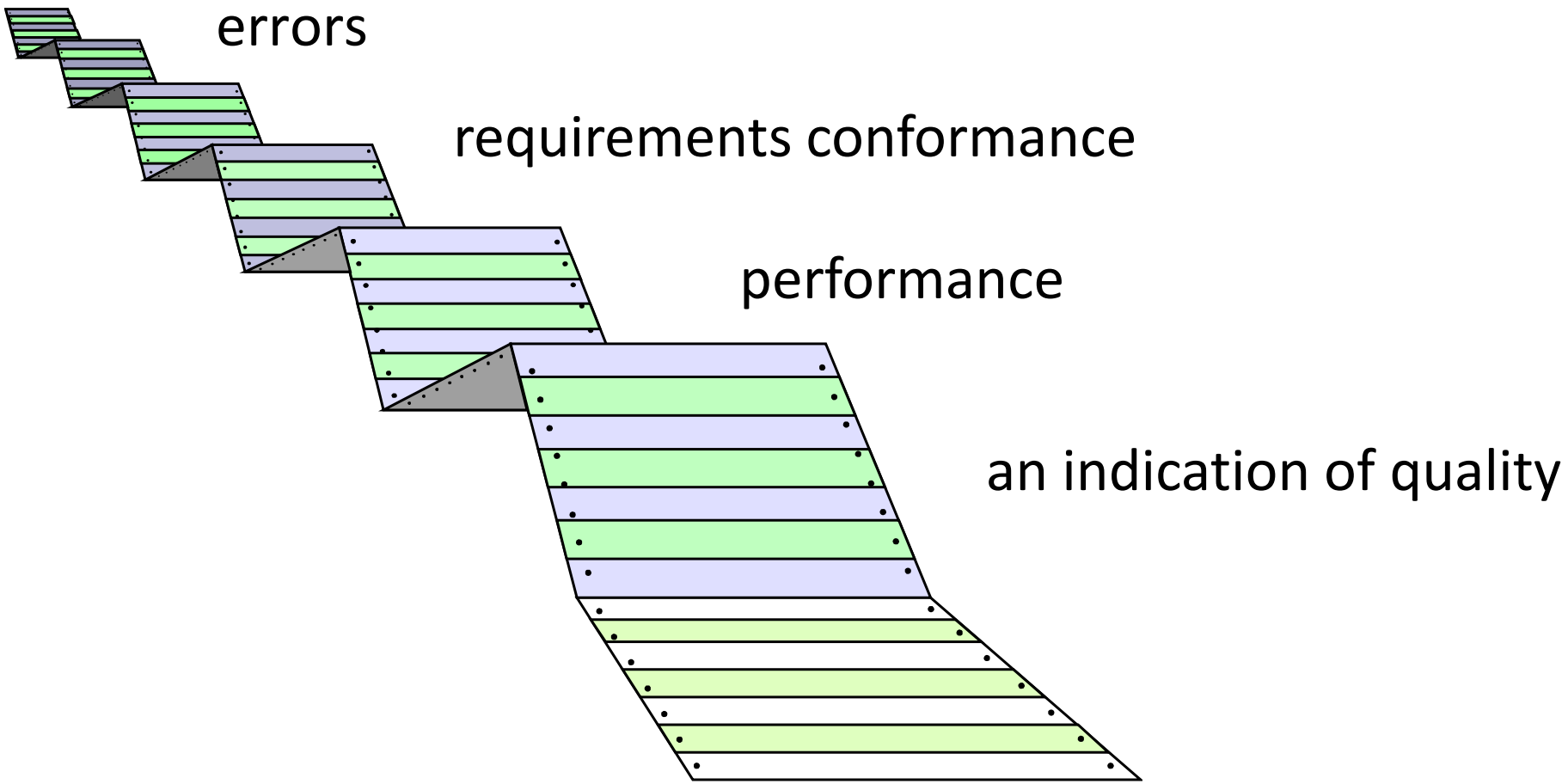
A good test should be best of breed

- Tests that have the highest likelihood of uncovering a whole class of errors should be used

A good test should be neither too simple nor too complex

- Each test should be executed separately; combining a series of tests could cause side effects and mask certain errors

WHAT TESTING SHOWS



Faults (Defects/Bugs)

- A fault is a defect in the program code that can cause incorrect operation
- Characteristics:
  - Static in nature (exists in the code)
  - May or may not lead to failures
  - Can be caused by coding errors, design mistakes, or requirement misunderstandings
- Some types of faults:
  - Algorithmic faults: Logic errors in program algorithms
  - Syntax faults: Violations of programming language rules
  - Computation faults: Incorrect mathematical calculations
  - Interface faults: Problems in component interactions

Erroneous States

- An internal program state that could lead to a failure
- Characteristics:
  - Dynamic in nature (occurs during execution)
  - Results from activation of a fault
  - May propagate through the system
- Components:
  - Data State Errors: Incorrect values in variables
  - Control Flow Errors: Wrong execution paths
  - Memory State Errors: Invalid memory conditions
  - Resource State Errors: Improper resource allocation

Failure

- External, observable incorrect behavior of the system
- Characteristics:
  - Visible to users or external systems
  - Results from erroneous states reaching system outputs
  - Can have varying degrees of severity
- Types of Failures:
  - Timing failures: Missing deadlines or incorrect timing
  - Output failures: Wrong results or responses
  - Crash failures: System stops functioning
  - Performance failures: System performs below requirements

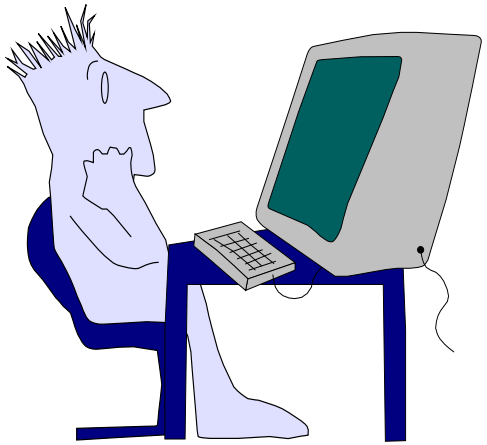
Relationship Between Concepts

- Fault → Erroneous State → Failure Chain: A fault in the code exists
- Fault gets activated under certain conditions
- Creates an erroneous state
- Erroneous state propagates
- Eventually leads to failure

Challenges in Software Testing

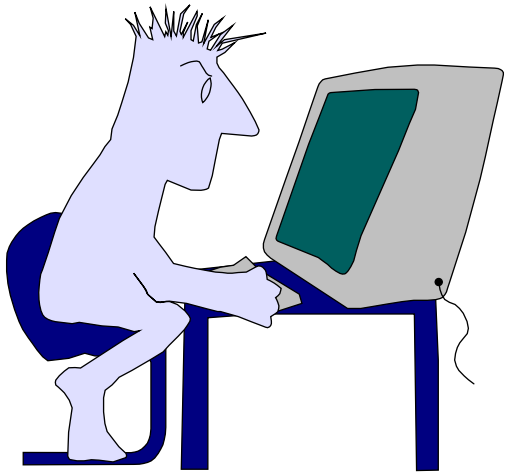
- Incomplete or changing requirements
- Limited resources
- Time constraints
- Difficulty reproducing defects.
- Additionally, testing complex systems can be challenging, as can testing for compatibility with various hardware and software configurations.

WHO TESTS THE SOFTWARE



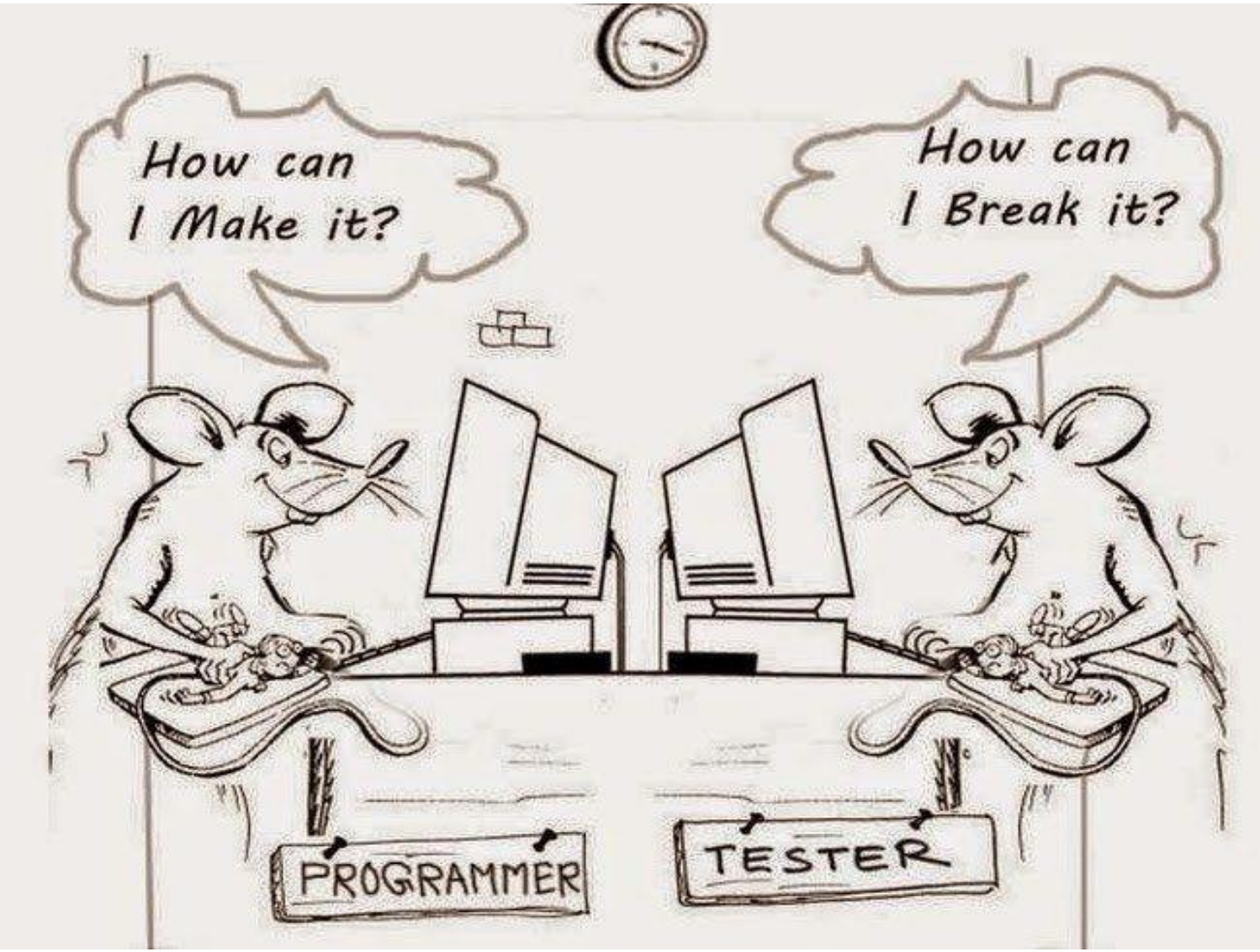
developer

Understands the system
but, will test "gently"
and, is driven by "delivery"



independent tester

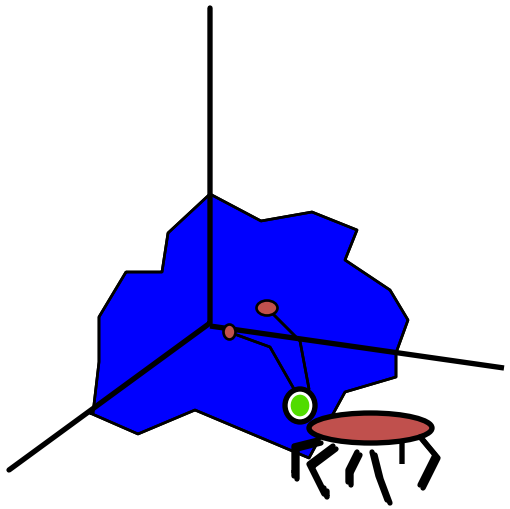
Must learn about the system,
but, will attempt to break it
and, is driven by quality


How can
I Make it?
How can
I Break it?
PROGRAMMER
TESTER

TEST CASE DESIGN

"Bugs lurk in corners and congregate at boundaries ..."

Boris Beizer



OBJECTIVE	to uncover errors
CRITERIA	in a complete manner
CONSTRAINT	with a minimum of effort and time

TEST CASE

- Test case is a **pre-defined set of instructions** addressing the steps to be taken in order to **determine** whether **the end product** exhibits the **desired outcome**. These instructions may include predefined **sets of inputs**, **conditions** along with **their respective end results**.
- IEEE defines test case as 'a set of input values, execution preconditions, expected results and execution post conditions, developed for a particular objective or test condition such as to exercise a particular program path or to verify compliance with a specific requirement.'

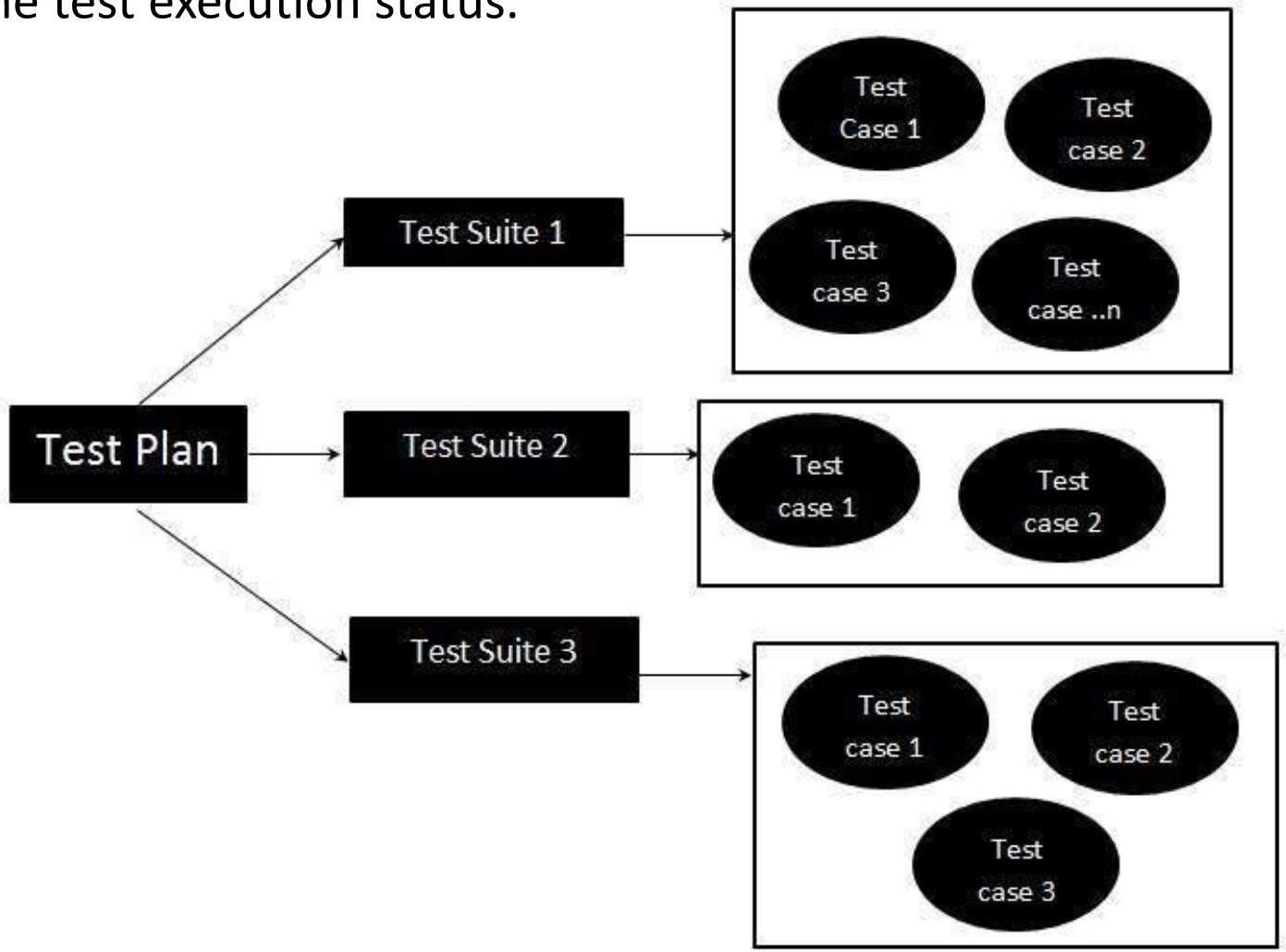
TEST CASE

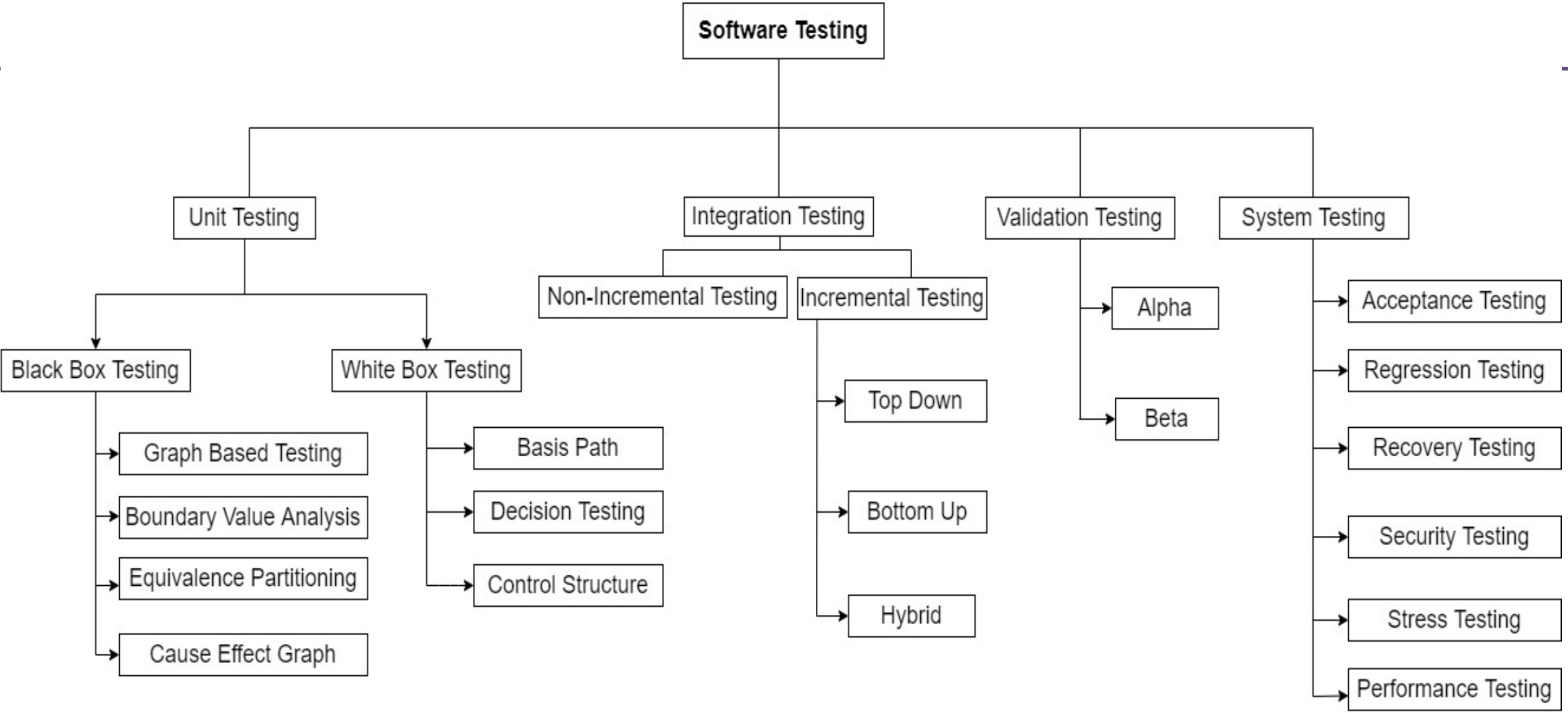
- However, in order to be through with one's testing one could often end up with too many test cases. To avoid such scenarios, one should find the **best test case design technique** as per ones requirement so as to reduce a significant number of test cases.
- These test case design techniques help create effective test cases covering the various features that determine the quality and value of a product.

https://www.educba.com/test-case-design-techniques/

TEST SUITS

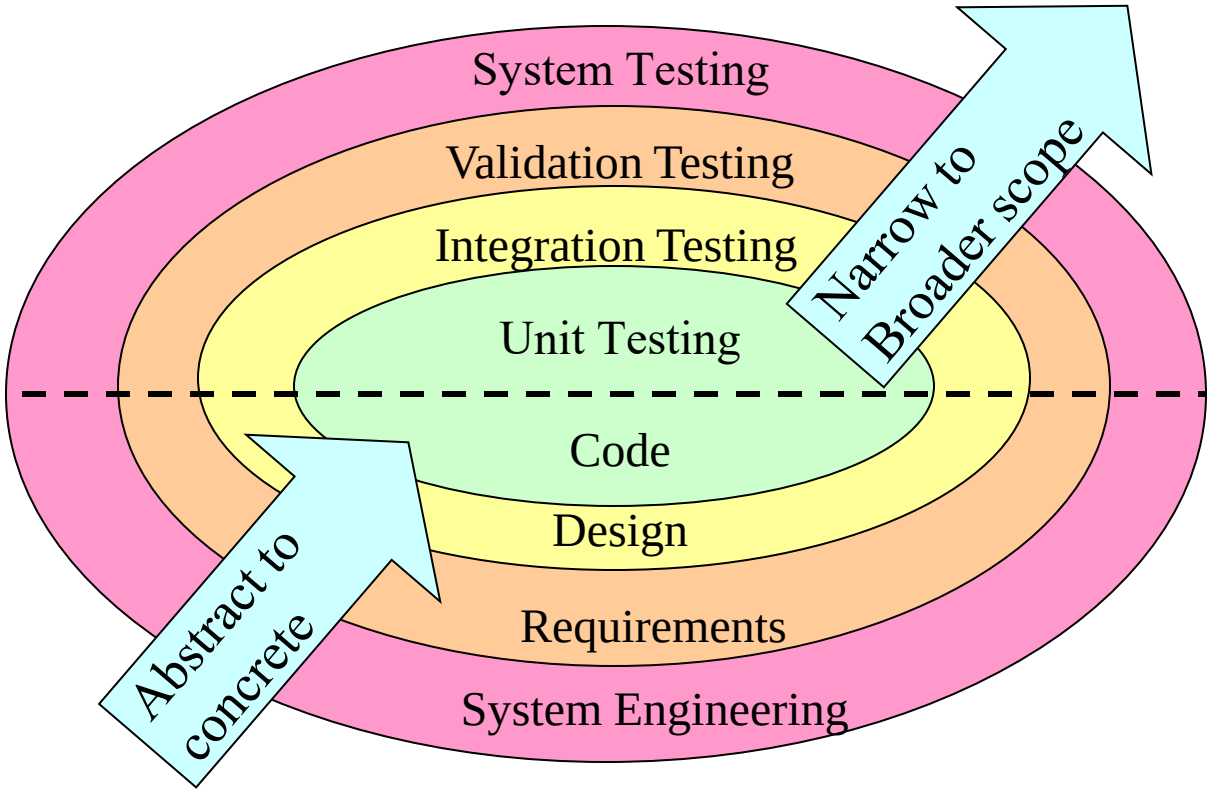
- Test suite is a container that has a set of tests which helps testers in executing and reporting the test execution status.





Note: White box testing and black box testing may not be necessary only part of Unit Testing. E.g. black box testing can be done in higher level such as validation testing or system level testing. Purpose here is to put under unit testing is that we can do both black and white box testing in – unit level.

A STRATEGY FOR TESTING CONVENTIONAL SOFTWARE



A STRATEGY FOR TESTING CONVENTIONAL SOFTWARE

- Unit testing
  - Exercises specific paths in a component's control structure to ensure complete coverage and maximum error detection
  - Components are then assembled and integrated
- Integration testing
  - Focuses on inputs and outputs, and how well the components fit together and work together
- Validation testing
  - Provides final assurance that the software meets all functional, behavioral, and performance requirements
- System testing
  - Verifies that all system elements (software, hardware, people, databases) mesh properly and that overall system function and performance is achieved

UNIT TESTING

- Focuses testing on the function or software module
- Concentrates on the internal processing logic and data structures
- Is simplified when a module is designed with high cohesion
  - Reduces the number of test cases
  - Allows errors to be more easily predicted and uncovered
- Concentrates on critical modules and those with **high cyclomatic complexity** when testing resources are limited

TARGETS FOR UNIT TESTING

- Module interface
  - Ensure that information flows properly into and out of the module
- Local data structures
  - Ensure that data stored temporarily maintains its integrity during all steps in an algorithm execution
- Boundary conditions
  - Ensure that the module operates properly at boundary values established to limit or restrict processing
- Independent paths (basis paths)
  - Paths are exercised to ensure that all statements in a module have been executed at least once

UNIT TESTING- SOME OF COMMON ERRORS

- Comparison of different data types
- Incorrect logical operators or precedence
- Expectation of equality when precision error makes equality unlikely (using == with float types)
- Incorrect comparison of variables
- Improper or nonexistent loop termination
- Failure to exit when divergent iteration is encountered
- Improperly modified loop variables
- Boundary value violations

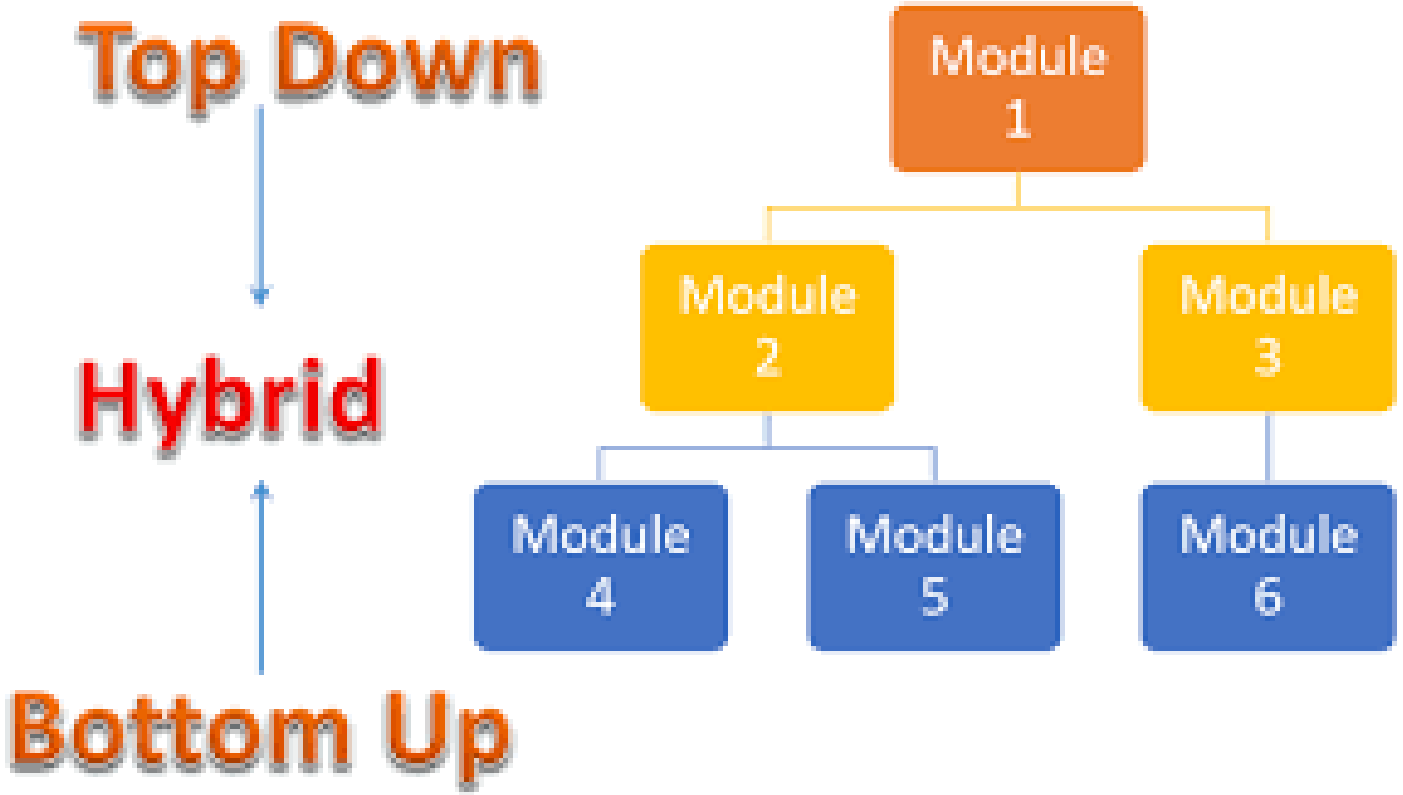
INTEGRATION TESTING

- Defined as a systematic technique for constructing the software architecture
  - At the same time integration is occurring, conduct tests to uncover errors associated with interfaces
- Objective is to take unit tested modules and build a program structure based on the prescribed design
- Two Approaches
  - Non-incremental Integration Testing
  - Incremental Integration Testing

INCREMENTAL INTEGRATION TESTING

- Three kinds
  - Top-down integration
  - Bottom-up integration
  - Hybrid approach
- The program is constructed and tested in small increments
- Errors are easier to isolate and correct
- Interfaces are more likely to be tested completely
- A systematic test approach is applied

INCREMENTAL INTEGRATION TESTING



Stubs and Driver

- Stubs and driver both are dummy modules and are only created for test purposes.
- Driver:
  - Driver are the ones, which are the “calling” programs
  - A component, that calls the unit under test
  - Drivers are used in bottom-up testing approach.
  - Controls the test cases
  - Drivers are dummy code, which is used when the sub modules are ready, but the main module is still not ready.

Stubs and Driver

- Stubs and driver both are dummy modules and are only created for test purposes.
- Stub:
  - A component, the unit under test depends on Partial implementation
  - Dummy modules that always used to simulate the low-level modules.
  - Stubs are used when sub programs are under construction.
  - Stubs are used in top-down testing approach, when you have the major module ready to test, but the sub modules are still not ready yet

Stubs and Driver

S.No	Stubs	Drivers
1.	Stubs are used in the Top down integration testing.	Drivers are used in the Bottom up integration testing.
2.	Stubs are also known as " called programs”	Drivers are also known as “calling program”
3.	Stubs are usually used in the unavailability of low-level modules.	Drivers are mainly used in place of high-level modules
4.	Stubs are used when lower-level of modules are missing and we need to test the main module.	Drivers are used when higher-level of modules are missing and we need to test the lower module.
5.	The test stub is a dummy program that integrates with an application to complete its functionality. It is relevant for testing that uses the top-down approach.	The test driver is a section of code that calls a software component under test. It is useful in testing that follows the bottom-up approach.

TOP-DOWN INTEGRATION

- Modules are integrated by moving downward through the control hierarchy, beginning with the main module
- Subordinate modules are incorporated in either a depth-first or breadth-first fashion
  - DF: All modules on a major control path are integrated
  - BF: All modules directly subordinate at each level are integrated
- Advantages
  - This approach verifies major control or decision points early in the test process
- Disadvantages
  - **Stubs** need to be created to substitute for modules that have not been built or tested yet; this code is later discarded
  - Because stubs are used to replace lower level modules, no significant data flow can occur until much later in the integration/testing process

BOTTOM-UP INTEGRATION

- Integration and testing starts with the most atomic modules in the control hierarchy
- Advantages
  - This approach verifies low-level data processing early in the testing process
  - Need for **stubs** is eliminated
- Disadvantages
  - **Driver** modules need to be built to test the lower-level modules; this code is later discarded or expanded into a full-featured version
  - **Drivers** inherently do not contain the complete algorithms that will eventually use the services of the lower-level modules; consequently, testing may be incomplete or more testing may be needed later when the upper-level modules are available

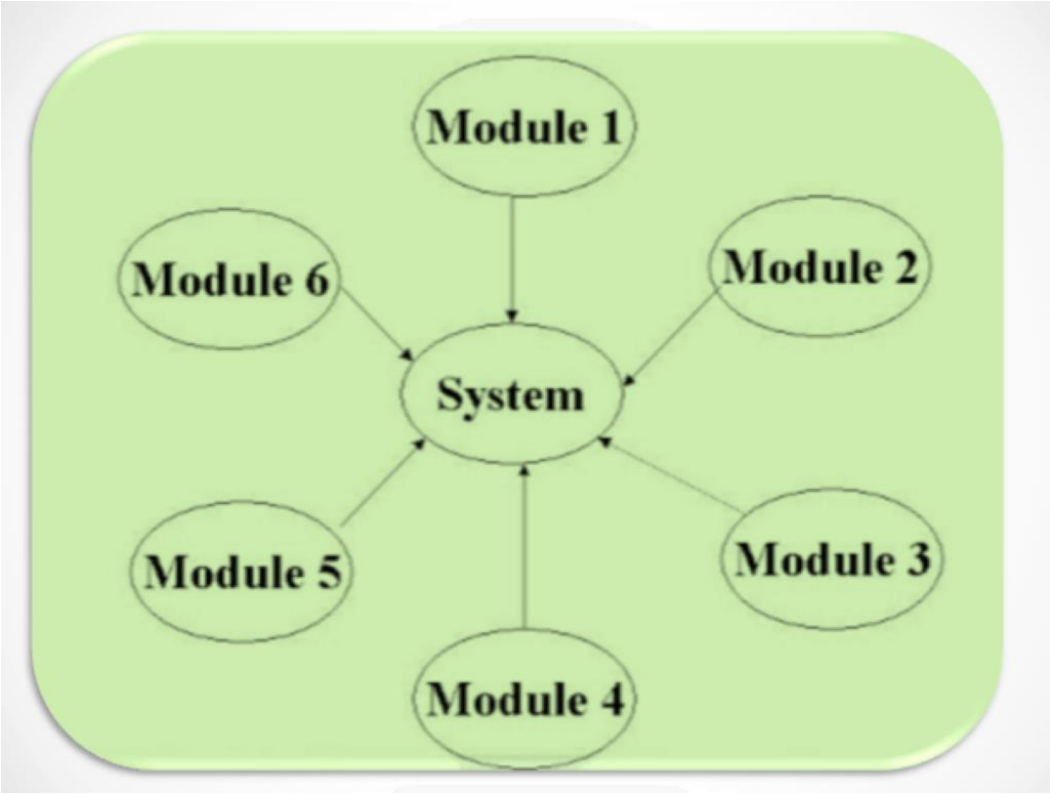
HYBRID

- Also known as “Sandwich integration testing” or “Mixed integration testing”
- Combination of top-down integration testing and bottom-up integration testing
- top modules are tested with the lower-level modules and lower-level modules are combined with the top-level modules and tested
- Uses both the concepts “**stubs**” and “**drivers**”. It is necessary to develop stubs and drivers for testing.

NON-INCREMENTAL INTEGRATION TESTING

- Commonly called the “Big Bang” approach
- All components are combined in advance
- The entire program is tested as a whole
- Chaos results
- Many seemingly-unrelated errors are encountered
- Correction is difficult because isolation of causes is complicated
- Once a set of errors are corrected, more errors occur, and testing appears to enter an endless loop

- There is also another concept of testing – big bang approach: all the components are integrated together at once and then tested as a whole.



Validation Testing

BACKGROUND

- Validation testing follows integration testing
- The distinction between conventional and object-oriented software disappears
- Focuses on user-visible actions and user-recognizable output from the system
- Demonstrates conformity with requirements
- Designed to ensure that
  - All functional requirements are satisfied
  - All behavioral characteristics are achieved
  - All performance requirements are attained
  - Documentation is correct
  - Usability and other requirements are met (e.g., transportability, compatibility, error recovery, maintainability)
- After each validation test
  - The function or performance characteristic conforms to specification and is accepted
  - A deviation from specification is uncovered and a deficiency list is created
- A configuration review or audit ensures that all elements of the software configuration have been properly developed, cataloged, and have the necessary detail for entering the support phase of the software life cycle

ALPHA TESTING

- Conducted at the developer's site by end users
- Software is used in a natural setting with developers watching intently
- Testing is conducted in a controlled environment

BETA TESTING

- Conducted at end-user sites(selected end-users)
- Developer is generally not present
- It serves as a live application of the software in an environment that cannot be controlled by the developer
- The end-user records all problems that are encountered and reports these to the developers at regular intervals
- After beta testing is complete, software engineers make software modifications and prepare for release of the software product to the entire customer base

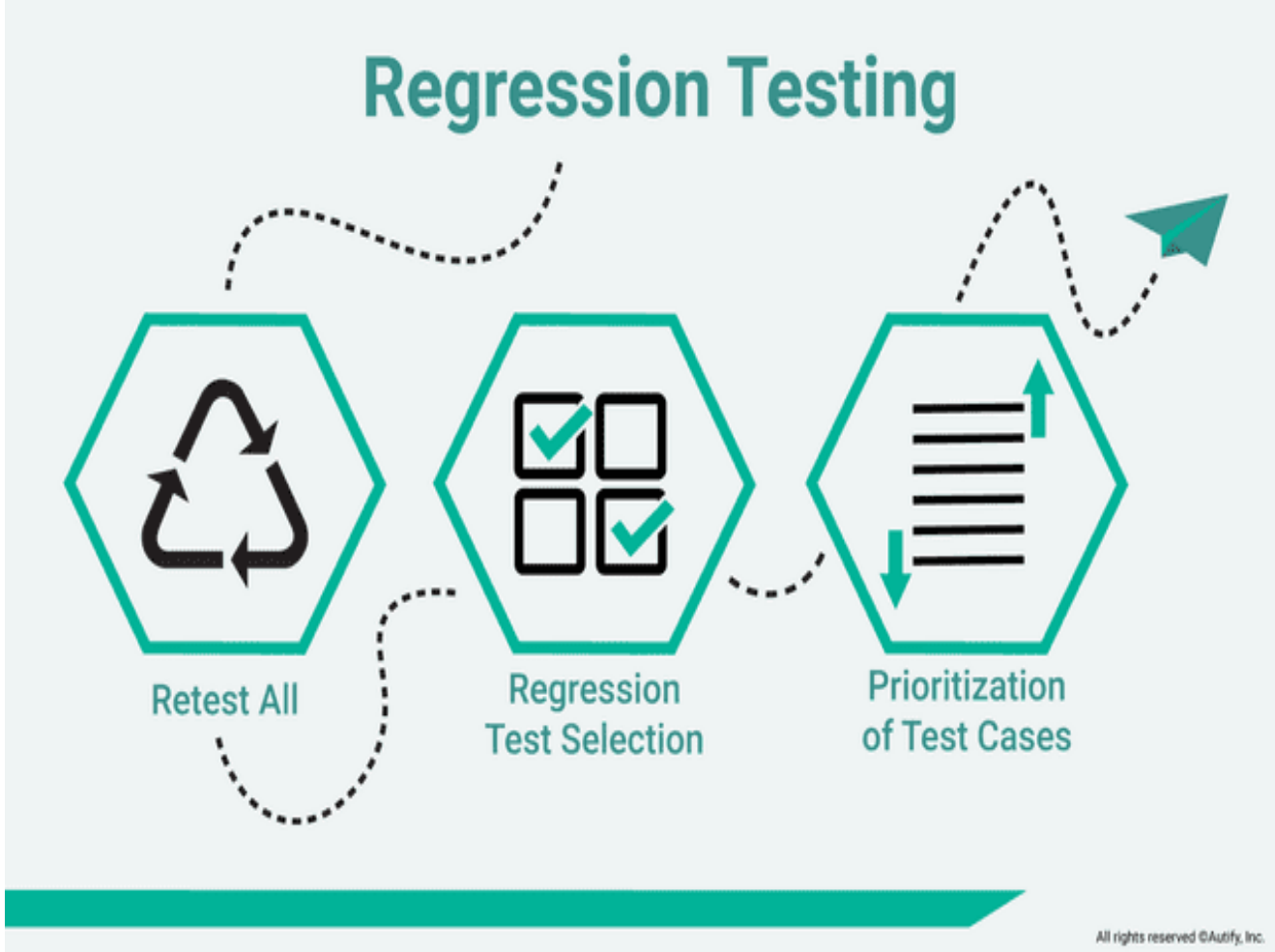
System Testing

REGRESSION TESTING

- Each new addition or change to baselined software may cause problems with functions that previously worked flawlessly
- Ensures that changes have not propagated unintended side effects
- Helps to ensure that changes do not introduce unintended behavior or additional errors
- May be done manually or through the use of automated tools

REGRESSION TESTING

- **Retest All**
  - All the test case suits should be re-executed.
  - The re-test is very expensive, as it requires enormous time and resources.
- **Regression Test Selection**
  - A selected test-case suit will execute rather than an entire test-case suit.
- **Prioritization Of Test Cases**
  - Prioritize the test case depending on business impact, critical and frequently functionality used.
  - Selection of test cases will reduce the regression test suite.



RECOVERY TESTING

- Tests for recovery from system faults
- Forces the software to fail in a variety of ways and verifies that recovery is properly performed
- Tests reinitialization, checkpointing mechanisms, data recovery, and restart for correctness
- Another example: When an application is receiving data from a network, unplug the connecting cable. After some time, plug the cable back in and analyze the application's ability to continue receiving data from the point at which the network connection was broken.

SECURITY TESTING

- Verifies that protection mechanisms built into a system will, in fact, protect it from improper access
- The goal of security testing is to:
  - To identify the threats in the system.
  - To measure the potential vulnerabilities of the system.
  - To help in detecting every possible security risks in the system.
  - To help developers in fixing the security problems through coding.
- Major focus areas in security testing:
  - Network Security
  - System Software Security
  - Client-side Application Security
  - Server-side Application Security
  - Data security

STRESS TESTING

- Stress testing is defined as the process of testing the hardware or software for its stability under a heavy load condition.
- impose abnormal input to stress the capabilities of the software.

- Input data volume, input data rate, processing time, utilization of memory, etc. are tested beyond the designed capacity.
- This testing is done to find the numerical point when the system will break (in terms of a number of the users and server requests etc.) and the related error handling for the same.

STRESS TESTING

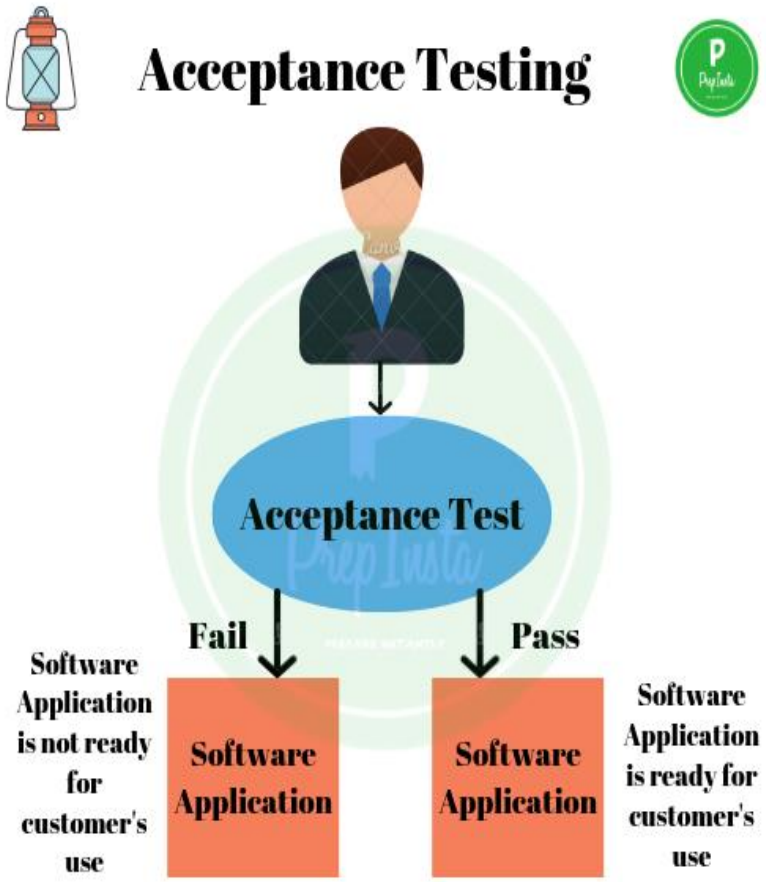
- Reasons:
  - To verify the system behavior under abnormal or extreme load condition.
  - To find the numerical value of users, requests etc., after which the system may break.
  - Handle the error graciously by showing appropriate messages.
  - To be well prepared for such conditions and take precautionary measures like code cleaning, DB cleaning, etc.
  - To verify data handling before the system breaks i.e. to see if data was deleted, saved or not etc.
  - To verify security threat under such breaking conditions etc.

PERFORMANCE TESTING

- Tests the run-time performance of software within the context of an integrated system
- Often coupled with stress testing and usually requires both hardware and software instrumentation
- Can uncover situations that lead to degradation and possible system failure

ACCEPTACE TESTING

- System testing performed by the customer himself:
  - to determine whether the system should be accepted or rejected.



END OF UNIT 06